RAPPORT DE DÉPLOIEMENT

Sécurité du Cloud — Stockage objet évolutif avec Swift

Application : SYNCORA v2

Démon C

Réalisé par	Encadrement
FOUDA DZOU AMBE Valdez KAMDOUM TEUFENG Josette	Mr. NGUIMBUS Emmanuel *Université Saint Jean — A.A. 2025-2026*

1. Introduction

Le présent rapport de déploiement décrit l'ensemble des procédures, configurations et vérifications nécessaires à la mise en production de la solution SYNCORA v2. Ce document couvre le déploiement du démon de synchronisation C, de l'interface web Angular, de l'API REST Node.js, ainsi que le déploiement complet via Docker Compose.

SYNCORA v2 est une solution de stockage objet évolutif basée sur OpenStack Swift, offrant une synchronisation bidirectionnelle automatique entre un système de fichiers Linux et un cluster Swift, complétée par une interface web permettant la gestion des fichiers, des conteneurs et du versioning des objets.

2. Prérequis

2.1 Infrastructure

Le déploiement requiert les éléments d'infrastructure suivants :

Composant	Spécification	Rôle
Machines Linux	Ubuntu 20.04+ / 22.04	Hôte du service SYNCORA
Cluster OpenStack Swift	OpenStack Yoga ou supérieur	Stockage objet distribué
Controller OpenStack	IP accessible sur le réseau	Auth Keystone + API Swift
RAM / CPU client	2 Go RAM min, 1 vCPU	Exécution du démon C
Connexion réseau	Port 5000 et 8080 ouverts	Keystone + API Swift

2.2 Logiciels requis

Logiciel	Version	Installation
GCC	11.x+	sudo apt install build-essential
Make	4.3+	sudo apt install make
python3-swiftclient	4.x	sudo apt install python3-swiftclient
python3-openstackclient	6.x	sudo apt install python3-openstackclient

3. Déploiement du démon C (SYNCORA v2)

3.1 Récupération du code source

```
# Cloner le dépôt
git clone
https://github.com/OpenSecureFoundation/Stockage-objet-evolutif-avec-Swif
t.git
```

3.2 Compilation

Le projet se compile via le Makefile fourni. Les quatre modules C sont compilés ensemble pour produire le binaire exécutable syncora :

```
# Nettoyer et compiler
make clean && make

# Vérifier que le binaire est bien généré
ls -lh syncora
# -rwxr-xr-x 1 user user 21K mar 18 20:05 syncora
```

3.3 Installation du binaire

```
# Copier le binaire dans un répertoire système accessible
sudo cp syncora /usr/local/bin/syncora
sudo chmod +x /usr/local/bin/syncora

# Vérifier l'installation
which syncora
# /usr/local/bin/syncora
```

3.4 Configuration du service systemd

Le fichier de service est disponible dans le dossier config/ du dépôt. Il doit être adapté avec les informations propres à l'environnement cible avant déploiement.

```
# Copier le fichier de service
sudo cp config/syncora.service.sample /etc/systemd/system/syncora.service

# Éditer le fichier de service
sudo nano /etc/systemd/system/syncora.service
```

Les paramètres suivants doivent obligatoirement être renseignés dans le fichier syncora.service :

Paramètre	Valeur à renseigner	Section

User	nom_utilisateur_linux	[Service]
Group	nom_utilisateur_linux	[Service]
OS_AUTH_URL	http://controller:5000/v3	Environment
OS_PROJECT_NAME	nom_du_projet	Environment
OS_USERNAME	utilisateur_openstack	Environment
OS_PASSWORD	mot_de_passe_openstack	Environment
WorkingDirectory	/chemin/vers/C_syncora_v2	[Service]

3.5 Configuration de la résolution DNS

Le controller OpenStack doit être résolvable par son nom d'hôte. Ajouter l'entrée suivante dans /etc/hosts :

```
# Éditer le fichier hosts
sudo nano /etc/hosts

# Ajouter la ligne suivante (remplacer par l'IP réelle) :
192.168.0.169  controller
```

3.6 Activation et démarrage du service

```
# Recharger la configuration systemd
sudo systemctl daemon-reload

# Activer le service au démarrage
sudo systemctl enable syncora.service

# Démarrer le service
sudo systemctl start syncora.service

# Vérifier le statut
sudo systemctl status syncora.service
```

Résultat attendu après démarrage :

● syncora.service - Service de Synchronisation Syncora Cloud (OpenStack)
Loaded: loaded (/etc/systemd/system/syncora.service; enabled; ...)
Active: active (running) since ...
=== SYNCORA V2 : DYNAMIQUE ET BIDIRECTIONNEL ===
--- Initialisation : /home/user/Documents/Syncora/ ---
[INIT] Dossier créé : documents

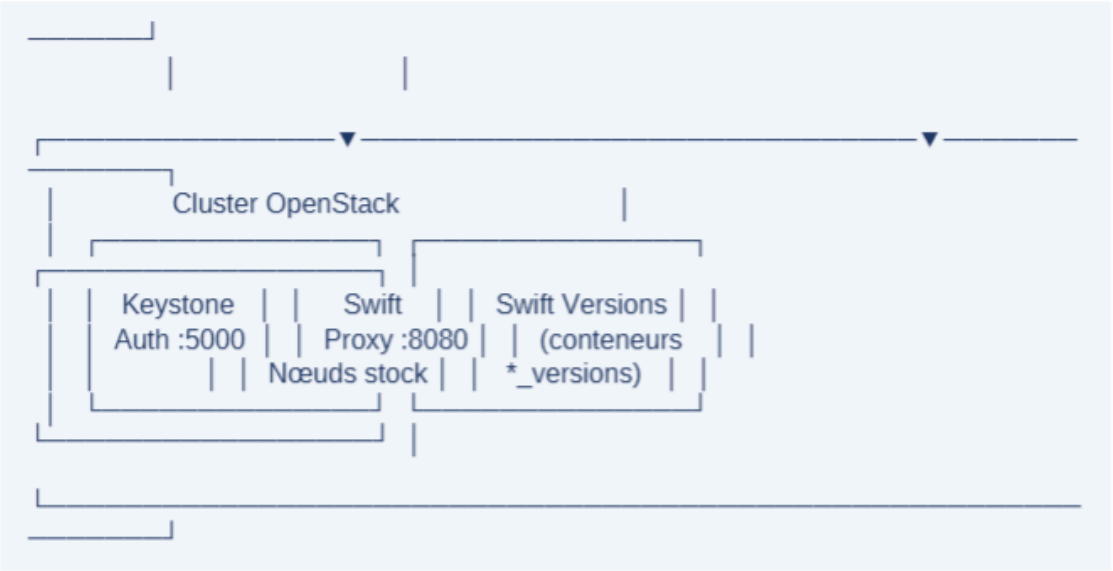
4. Problèmes connus et solutions

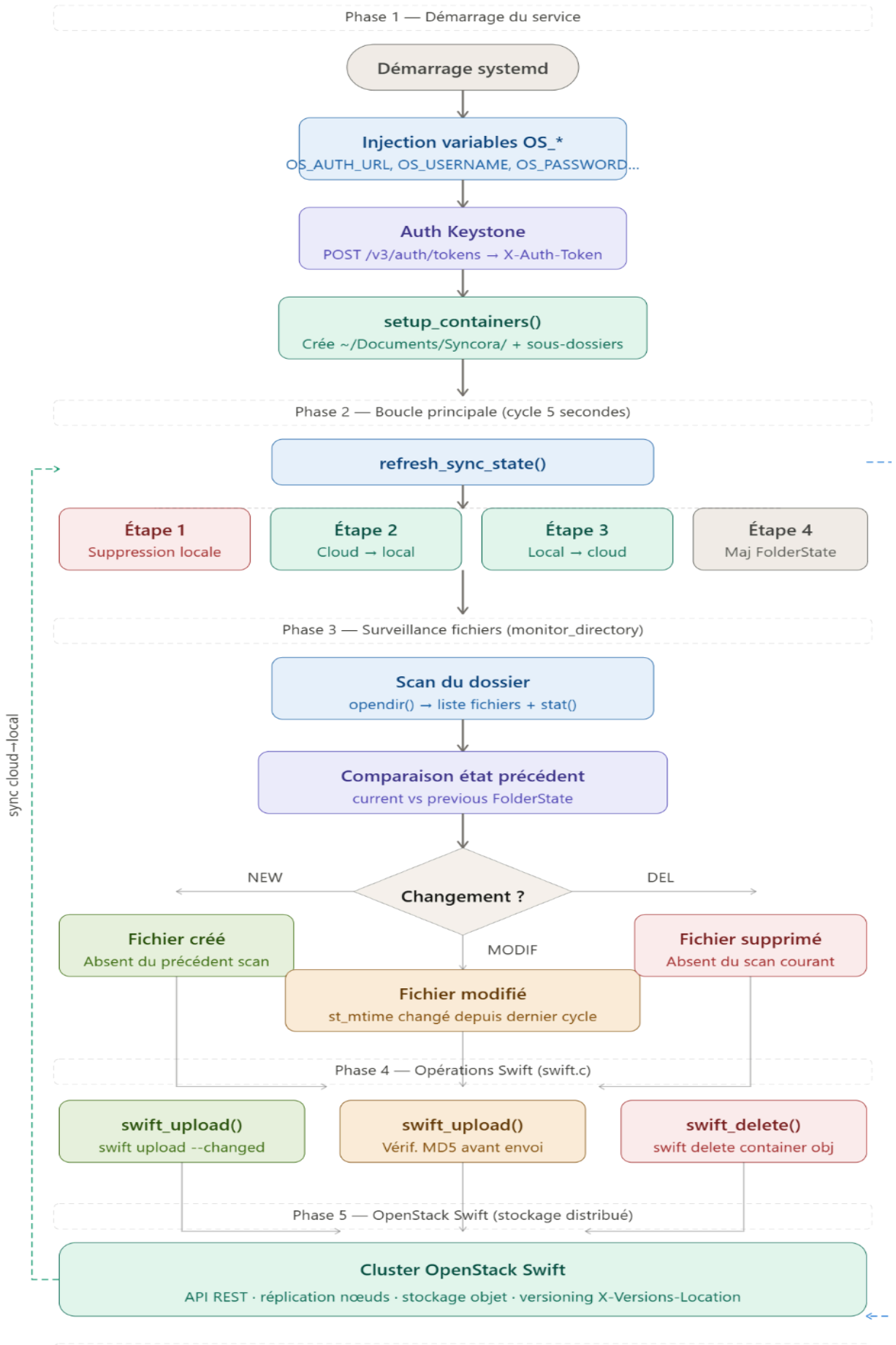
Problème	Cause probable	Solution
Service ne démarre pas	Variables OS_* incorrectes dans syncora.service	Vérifier le fichier de service, relancer systemctl daemon-reload
Erreur de connexion Swift	controller non résolvable ou port 5000 fermé	Vérifier /etc/hosts et la connectivité réseau vers le controller
Token expiré (401)	Token Keystone périmé (durée de vie ~1h)	Redémarrer le service : sudo systemctl restart syncora.service
Fichiers non synchronisés	MAX_FILES (1024) atteint dans un dossier	Réduire le nombre de fichiers ou augmenter MAX_FILES dans monitor.h

5. Architecture de déploiement et workflow

Le schéma suivant représente l'architecture complète du déploiement SYNCORA v2 en production :

ARCHITECTURE DE DÉPLOIEMENT COMPLÈTE




Phase 1 — Démarrage du service
Démarrage systemd
Injection variables OS_*
OS_AUTH_URL, OS_USERNAME, OS_PASSWORD...
Auth Keystone
POST /v3/auth/tokens → X-Auth-Token
setup_containers()
Crée ~/Documents/Syncora/ + sous-dossiers
Phase 2 — Boucle principale (cycle 5 secondes)
refresh_sync_state()
Étape 1
Suppression locale
Étape 2
Cloud → local
Étape 3
Local → cloud
Étape 4
Maj FolderState
Phase 3 — Surveillance fichiers (monitor_directory)
Scan du dossier
opendir() → liste fichiers + stat()
Comparaison état précédent
current vs previous FolderState
Changement ?
NEW
DEL
MODIF
Fichier créé
Absent du précédent scan
Fichier supprimé
Absent du scan courant
Fichier modifié
st_mtime changé depuis dernier cycle
Phase 4 — Opérations Swift (swift.c)
swift_upload()
swift upload --changed
swift_upload()
Vérif. MD5 avant envoi
swift_delete()
swift delete container obj
Phase 5 — OpenStack Swift (stockage distribué)
Cluster OpenStack Swift
API REST · réplication nœuds · stockage objet · versioning X-Versions-Location
sync cloud→local

Le workflow complet de SYNCORA v2 est organisé en 5 phases,

Phase 1 — Démarrage : systemd injecte les variables OpenStack, Keystone génère le token d'accès, puis `setup_containers()` crée l'arborescence locale.

Phase 2 — Boucle principale : `refresh_sync_state()` s'exécute toutes les 5 secondes et synchronise l'état des conteneurs en 4 étapes ordonnées (suppression prioritaire → cloud→local → local→cloud → mise à jour FolderState).

Phase 3 — Surveillance fichiers : `monitor_directory()` scanne chaque dossier, compare avec l'état précédent, et branch sur 3 cas : fichier nouveau (NEW), modifié (MODIF via `st_mtime`), ou supprimé (DEL).

Phase 4 — Opérations Swift : les 3 fonctions de `swift.c` exécutent les commandes CLI correspondantes, avec vérification MD5 via `--changed` pour éviter les uploads inutiles.

Phase 5 — OpenStack Swift : le cluster distribué reçoit les objets, les réplique sur les nœuds, et gère le versioning via `X-Versions-Location`.

Les deux flèches en pointillés sur les côtés représentent les boucles : le `sleep(5)` à droite (retour au début du cycle) et la synchronisation cloud→local à gauche (nouveaux conteneurs détectés).

6. Conclusion

Le déploiement de SYNCORA v2 couvre deux composantes complémentaires : le démon de synchronisation C intégré au système via systemd, Le mode de déploiement manuel pour le développement permettent une mise en service.